

GeoCadastra vs BhoomiDrishti AI

Comparison and independent verification - 14 September 2026

Part I — Verification verdict

This report was independently audited before publication. Every factual claim in Part II that could be checked by running code, counting files, or reading source was checked.

****Result:** 22 of 22 verifiable claims are true. No claim was found to be false, and no claim was found to be materially overstated.****** Two claims deserve added nuance rather than correction; both are noted below and in the relevant sections.

The two most consequential claims were reproduced from scratch rather than accepted:

- The alleged area-tolerance defect in `geocadastra/core/capacity.py` is ****real and reproduces exactly as described.**** A 100 m² block with three records of 34 m² and ± 1 m² tolerance has a feasible allocation of 33/33/34. Starting from a 20/20/60 arrangement, the solver reports `recorded_areas_not_met_after_snapping` with `max_error_m2: 2.0` while its own optimiser reports "Optimization terminated successfully". Starting from the already-feasible arrangement, the same function accepts it. The cause is as diagnosed: the equality constraint targets exact recorded areas, so tolerance is only checked after the fact, never used to distribute slack across records.
- The claim that BhoomiDrishti's model metrics are computed from a name hash rather than from images is **real and reproduces exactly**. Running its `computeModelMetrics()` verbatim on a patch that does not exist, `file_that_does_not_exist`, returns IoU **0.883** and an inference time of **197 ms** — the exact figures quoted in Part II. No image, mask or weight file is read anywhere in that path.

Nuance one. The capacity defect is a **failure to find a solution that exists**, not a failure that emits a wrong answer. In every reproduced case the system declined, `recorded_area_refinement_unresolved`, and left the input graph untouched. It is a real limitation and should be fixed, but it fails safe rather than silently certifying an incorrect allocation.

Nuance two. "No dedicated frontend found" is accurate for GeoCadastra, but the absence was a stated scope decision in its build plan ("Backend and logic only, no UI"), not an oversight. The practical consequence in Part II stands regardless: a backend that cannot be seen is hard to demonstrate.

Part II — GeoCadastra and BhoomiDrishti AI: what is actually built

Comparison date: 14 September 2026. Your project is the [geocadastra/](#) Python application at commit [87b14c2](#). The comparison project is [new_project_of_someone_else/BhoomiDrishti_AI/](#), whose checked-out commit is [12cf65c](#).

This comparison follows source code and data files. A label on a page, a dependency in a package file, and a completed operation are different kinds of evidence. Statements about BhoomiDrishti apply to the supplied folder; they do not establish what its author might have implemented elsewhere.

The central difference

GeoCadastra implements methods for constructing a parcel map, keeping neighbouring parcels consistent, checking recorded areas, combining boundary evidence, saving edits, and reconstructing history. It has actual model-training and prediction code, plus separate calibration and survey-planning modules. Its connected workflow currently operates on generated test wards and simulated boundary evidence. It has no dedicated browser application in this repository.

BhoomiDrishti implements a browser application with a national map, image-patch browser, parcel explorer, topology display, model-comparison screen, field screen, export screen, and billing screen. Its map rendering, filtering, image display, some polygon operations, and parcel-explorer downloads are real implementations. However, the supplied application does not execute the model

training or inference shown on those screens. Several important results are produced by fixed constants, arithmetic on names, random numbers, and timers.

Your project has substantially more of the underlying parcel-processing software. BhoomiDrishti has substantially more of the interface through which a person could inspect and demonstrate a future service. Neither supplied project establishes a complete, validated service for real cadastral surveys.

What your project is trying to do, in ordinary language

A parcel is a piece of land represented by a boundary and associated with a record. A building outline is a different object: a building can sit inside one parcel, occupy only part of it, or cross a parcel boundary. Recognising roofs in a photograph therefore does not, by itself, recover

land records.

Your project addresses a harder problem: given a block of land, recorded parcel areas, approximate

parcel locations, an older map, visible boundary clues, and some measured points, construct a consistent map and identify what remains uncertain.

Consider a 1,000 m² block with two records of 400 m² and 600 m². An image may show part of their

separating wall, while trees hide the rest. The image constrains where the visible part should go.

The recorded areas constrain how much land each parcel should receive. An older map provides another suggested location. Survey measurements can constrain particular corners.

These constraints narrow the possible answers, but do not always select a unique boundary.

Multiple boundaries can enclose the same areas. Your software can produce a candidate, reject unsafe moves, and report unresolved constraints. Matching an area is not proof that a boundary is correct, and neither project verifies legal ownership against an authoritative registry.

Feature comparison

Capability	GeoCadastra	BhoomiDrishti in the supplied folder
Browser interface	No dedicated frontend found	React application with seven main pages and map/detail views
Input used by the current processing workflow	Generated ward, stored records, simulated boundary raster	Bundled images and GeoJSON, selected in the browser
Actual image files	Synthetic rasters generated by code	690 image PNGs, 690 mask PNGs, 690 binary PNGs
Train a neural network	Implemented in PyTorch	Training messages displayed by a timer
Run a neural network	Standalone prediction implementation; not connected to the ward worker	No executing model path found; model scores generated from names and settings
Use parcel areas to place boundaries	Raster assignment plus shared-node area refinement	No comparable recorded-area solver found
Shared boundaries	One edge referenced by neighbouring faces	Independent GeoJSON polygons
Geometry validation	Actual validity, overlap, area and block-coverage checks	Some Turf operations; listed anomalies largely generated from presets
Durable parcel storage	PostgreSQL/PostGIS tables, IDs, versions and relationships	Server arrays and browser state; database schema has no application tables
Correct a measured corner	Validated changeset and saved survey measurement	Verification action changes a status field

Capability	GeoCadastra	BhoomiDrishti in the supplied folder
Accuracy reporting	Computed metrics and a tracked failing model target	Fixed or formula-generated confidence and model scores
Calibration of position uncertainty	Implemented separately; not connected to ward API	No corresponding implementation found
Decide survey visit order	Implemented and tested in simulation	Displayed queues and preset risk labels
Resume processing	Per-block stored status and real task implementation	Local progress timers; server run records have no processing worker
Outputs	Spatial parcel queries, conflict/constraint reports, actual vector-tile bytes	Real CSV/GeoJSON downloads in parcel explorer; export-center jobs return filenames
Tests	Python tests, including database and acceptance tests	No application test suite or test script found in supplied source

Stage 0 — Make test wards for which the answer is known

Your Stage 0 creates a small artificial place. It generates roads, divides the place into blocks, divides blocks into parcels, places buildings and vegetation, and renders a colour image plus height rasters. It also produces the exact parcel geometry used to make those images.

The generator uses three layout styles: formal strips, more irregular informal parcels, and larger institutional parcels. It deliberately makes some parcel boundaries invisible. It can also create buildings crossing parcel boundaries. That prevents the tests from assuming that every roof outline

is a land boundary.

It then creates an imperfect older map by shifting and perturbing geometry, dropping records, and merging some neighbouring records. Sparse survey points provide another source of information.

Because the true map is known, later stages can measure how well they recover it.

The random seed and parameters make a ward reproducible. Generator version 2 collects subdivision

cuts into one intersected line network and constructs parcels from that network. This avoids the earlier thin gaps caused by constructing and rounding child polygons independently. Older stored jobs preserve their generator version when their raster evidence is regenerated.

There are limits to this test environment. The generated images are simplified, the store uses a fixed working coordinate system, and synthetic ingestion supplies parcel seed locations from the generated geometry. The generator's parcels cover the land beneath rendered road surfaces rather

than implementing a complete road-right-of-way ownership model. Success here is useful engineering

evidence, but not real-world survey validation.

BhoomiDrishti takes a different approach. It includes 690 named image patches and matching mask/binary files. Every manifest entry has all three files, confirmed by counting. The example inspected consists of an aerial-image patch, a coloured category mask and a binary category image.

The application labels these assets as SVAMITVA data; their external provenance was not independently established in this comparison.

It also generates display parcels. `generateMicroParcels()` creates random rectangles, identifier-like strings, owner names, ownership categories and confidence values. The explorer requests 300 such parcels. The static `parcels_21.geojson` also contains 300 polygons with this kind of property structure. These are not the same dataset as the six sample parcels returned by its API.

The distinction is the role of synthetic data. Your generator creates inputs and a known answer for testing a reconstruction method. BhoomiDrishti generates much of the parcel information that the interface displays as a result.

Sources: your generator (`geocadastra/synth/generator.py`), shared subdivision
(`geocadastra/synth/subdivision.py`), BhoomiDrishti parcel generator
(`artifacts/bhoomi-ai/src/utils/parcelGenerator.ts`), parcel explorer
(`artifacts/bhoomi-ai/src/components/ParcelExplorerModal.tsx`).

Stage 1 — Represent neighbouring parcels using the same boundary

Your geometry representation has three parts: nodes are boundary junctions or endpoints; edges are

lines between nodes; faces are the enclosed land regions. A face refers to its edges rather than storing a separate copy of every coordinate. Interior rings are supported, so a parcel can surround a hole or another parcel.

Suppose parcels A and B share a boundary. In your graph, both refer to the same edge. Moving an endpoint changes the geometry derived for both faces. This removes the need to remember to edit two

independent copies of the same boundary.

Before creating that graph, your code brings nearly coincident line vertices together, intersects lines so crossings become explicit, places geometry on a precision grid, and constructs closed

faces. The working graph uses a 1 mm coordinate grid. That is a numerical representation choice; it

does not mean the survey is accurate to 1 mm.

The block-building code uses road lines and a ward boundary to construct blocks. Authoritative boundary linework acts as a fixed reference during this operation. Geometry wrappers also carry coordinate-system information and reject incompatible combinations instead of silently treating degrees and metres as interchangeable.

BhoomiDrishti displays independent GeoJSON polygons with Leaflet and performs some operations with

Turf. That is actual geospatial code. Its topology screen loads the 300-polygon file, takes a city-dependent subset, translates polygons around a selected city, rotates and scales them, and displays the result. If the file cannot supply features, it can generate a square grid.

Its displayed error list is not the output of a comprehensive topology validator. City definitions prescribe anomaly counts. The code chooses error categories and risk values from templates. An overlap calculation is attempted for one category, but an overlap entry can still be created when the intersection is empty. Other categories can be listed without actually checking for the stated geometric defect.

Its auto-fix attempts to union the first two polygons, marks the second as hidden, then clears all listed errors and reports all of them fixed. It does not walk the reported errors, repair each one, and recheck the resulting parcel fabric. Joining two parcels also changes their identity, which is not a safe general replacement for fixing a shared boundary.

Sources: your graph (`geocadastra/core/graph.py`), line preparation (`geocadastra/core/planarize.py`), block construction (`geocadastra/core/blocks.py`), BhoomiDrishti topology (`artifacts/bhoomi-ai/src/components/TopologyModule.tsx`).

Stage 2 — Save the map, protect edits, and retain evidence

Your store uses PostgreSQL with PostGIS geometry columns. It stores nodes, edges, faces, face-boundary references, changesets and evidence records. Later additions store wards, blocks, recorded parcels, older-map geometry, survey points, alignment records and processing conflicts.

A record's identity is separate from the position of a polygon in a list. One recorded parcel can own multiple face components. Survey points have a database relationship tying the parcel and ward

together; a point cannot simply refer to a parcel in another ward.

A changeset represents one group of edits. The software loads the graph and its versions, performs

proposed moves, checks for concurrent modification, validates the resulting geometry, and saves the

change transactionally. Unsafe changes are refused. Checks include invalid faces, overlaps, worsening recorded-area errors and worsening block coverage.

The shared-edge representation and database checks solve different problems. Sharing edges keeps

adjacent representations connected; validating the result prevents a connected graph from being moved into an invalid or unacceptable shape. The low-level graph object alone is not a substitute for the validated database edit path.

Your evidence log includes a SHA-256 hash chain and validation code. New-format history also records enough initial topology and subsequent events to reconstruct a block without relying on its

current face cache. This is useful for explaining how a result was obtained. The hash chain is not an independently signed certificate and does not prove that a measurement itself was truthful.

BhoomiDrishti's server stores its sample regions, parcels and processing runs in module-level arrays. Parcel edits mutate those objects. These changes disappear when that server process restarts. The browser additionally keeps temporary screen-to-screen summaries in `sessionStorage`.

Its database package contains connection setup, but the schema file exports no application tables, and the inspected parcel routes do not use a database. Having a database library installed does not

provide durable storage for those routes.

Its recent-activity and chain-of-custody text is therefore not equivalent to your event history.

The supplied export screen displays an Ed25519 signing label, but no matching signing implementation was found in the application path.

Sources: your schema (`geocadastra/store/schema.py`), changesets (`geocadastra/store/changeset.py`), evidence log (`geocadastra/store/provenance.py`), history reconstruction (`geocadastra/store/replay.py`), BhoomiDrishti routes (`artifacts/api-server/src/routes/bhoomi.ts`), its database schema (`lib/db/src/schema/index.ts`).

Stage 3 — Allocate the block among known parcels

This is one of the main distinctions in your project. The inputs are a block polygon, recorded parcel areas, approximate parcel seed points, and an image-sized field describing where boundaries

are likely to lie.

The first algorithm divides the raster into small neighbouring regions of pixels. It builds a graph of those regions. Crossing a strong boundary clue costs more than crossing a weak one. It then measures the cost of reaching each region from each parcel's seed and solves an allocation problem

using the region areas and the recorded parcel areas.

The allocation is converted into one parcel label per small region, and those labels become polygons. There are explicit handling paths for missing areas or seed points, mismatched totals, and parcels receiving no area. A simpler comparison algorithm omits the recorded-area constraint, allowing tests to examine what the areas contribute.

An exact continuous allocation does not automatically become exact polygon areas after assigning

whole pixel regions. The transport code also rescales its internal demand when needed to balance the optimisation problem and records discrepancies. It does not rewrite the original stored parcel-area records to make its answer look correct.

The newer `capacity.py` then refines the shared geometry. It tries to move graph nodes towards the

recorded areas while penalising displacement. Nodes supported by stronger simulated evidence get a

larger movement penalty. Block corners remain fixed; other exterior nodes can move along their boundary segment. The candidate is snapped to the coordinate grid and checked for valid faces, overlaps, block coverage and each recorded area.

This is a bounded local solver, not a general solution for every possible collection of records. It declines cases over 256 optimisation variables and can decline infeasible or unsupported arrangements. The worker records unresolved refinement rather than claiming success.

There is also a reproduced defect still present in this checkout. In a 100 m² block, records of 34/34/34 m² with a tolerance of ± 1 m² have a feasible allocation of 33/33/34 m². Starting from 20/20/60 m², the solver forces its first two equality targets to 34, leaves 32 for the third, and rejects the candidate. Starting from the feasible arrangement, the same function accepts it. The optimisation needs to represent tolerance intervals, rather than checking them only after forcing exact targets. Note that the failure mode is a refusal, not a false acceptance.

BhoomiDrishti has no corresponding area-allocation or recorded-area refinement code in the supplied

application. Its generated parcel areas describe generated rectangles. Its topology display

transforms existing polygons, and its inference screen computes a parcel count from the characters

in a patch name. Neither operation uses recorded areas to solve for land boundaries.

Sources: `your allocation (geocadastra/core/transport.py)`, `node refinement (geocadastra/core/capacity.py)`, `stored-area checks (geocadastra/store/constraints.py)`, `BhoomiDrishti handlers (artifacts/bhoomi-ai/src/App.tsx)`.

Stage 4 — Learn image clues and predict them on another image

Your project contains an actual PyTorch network. It accepts the colour image and height above terrain, calculated as DSM minus DTM. DSM is the height of the visible surface, including objects; DTM is ground elevation. Subtracting them gives a measure of how far objects extend above the terrain.

The network processes image and height inputs separately at first, lets their reduced-resolution features exchange information, passes the result through a shared feature extractor, and produces

several outputs: distance to parcel boundaries, estimated uncertainty in that distance, road pixels, building pixels, and layout/land-use classes.

The source calls the boundary output an SDF. Its training target is actually a nonnegative distance to the nearest parcel boundary, not a positive/negative inside-versus-outside distance. Likewise, the current land-use labels are primarily generated settlement-style classes plus roads, not a validated catalogue of legal land uses.

Training computes errors against generated targets, calculates gradients, updates model weights, and records losses. Checkpoints save model weights, optimiser and learning-rate state, random state

and completed epochs so training can resume. Inference divides a larger raster into overlapping tiles, runs the model, and blends predictions, including disagreement between tiles in the uncertainty calculation.

This stage is implemented, but its accuracy target remains unmet in the tracked CPU-scale test. The

ward-processing worker also does not call this inference function. It uses a simulated boundary-evidence field instead. Thus, a ward job finishing successfully is not proof that the trained model successfully reconstructed that ward.

BhoomiDrishti's image viewer loads the bundled PNG images, masks and binary images. Its training

screen advances with a timer and displays predefined losses and accuracy messages. Its model list

contains seven named checkpoint choices, but the inspected code does not load seven trained models.

`computeModelMetrics()` computes IoU, Dice, precision, recall, F1, loss and inference time from a patch-name hash, the model label, epoch count and resolution. The comparison viewer reuses the same

saved mask and changes its brightness, contrast and saturation to make the displayed model outputs

look different.

Those metric functions were executed after removing only their TypeScript number/string annotations. A nonexistent patch named `file_that_does_not_exist` still received IoU 0.883 and an inference time of 197 ms. No image, target mask or model weights were loaded to produce that result. These numbers therefore cannot be compared to your measured model performance.

Its screen also calls a binary category PNG a DTM layer. Displaying categories in two colours does not calculate terrain elevation in metres. The corresponding displayed height difference is a predefined message, not a subtraction of measured height rasters.

Sources: your network (`geocadastra/models/backbone.py`), training targets
(`geocadastra/models/dataset.py`), training (`geocadastra/models/train.py`), tiled inference
(`geocadastra/models/infer.py`), accuracy test (`geocadastra/tests/test_stage4_acceptance.py`),
BhoomiDrishti metrics (`artifacts/bhoomi-ai/src/App.tsx`).

Stage 5 — Combine boundary suggestions and preserve disagreements

Your fusion code represents each available source as a suggested position and an uncertainty. The sources can include the older map, nearby survey measurements, and the model's boundary prediction.

Missing evidence stays missing rather than becoming a fabricated observation.

When sources are sufficiently consistent, their positions are combined with greater weight given to

the source claiming lower positional uncertainty. When they disagree beyond a configured threshold,

the code records a conflict instead of averaging away the disagreement. The output records which sources contributed.

Boundary rules constrain proposed movements. Established block corners remain pinned. Other boundary nodes may slide along permitted boundary portions. Proposed moves still go through

changeset validation before being persisted; a numerical average is not automatically an acceptable

parcel map.

In the current ward worker, this stage combines saved older-map boundaries and the survey points

reserved for fusion. Model outputs are not supplied. The alignment endpoint fits and stores an affine transformation from paired control points, applies it to original older-map geometries, and carries its residual into the older-map uncertainty. This implementation does not constitute a complete uploaded-raster alignment service.

BhoomiDrishti displays image, mask and binary layers, but no corresponding multi-source boundary

estimator was found. It does not read saved survey measurements and recorded parcel constraints to

compute an uncertainty-weighted shared-node position. Its completion marker is randomly placed inside a selected state's bounding box; it is not a location recovered from the image's georeferencing.

Sources: your fusion (`geocadastra/core/fusion.py`), conflicts (`geocadastra/core/conflicts.py`), worker (`geocadastra/jobs/orchestrator.py`), alignment endpoint (`geocadastra/api/main.py`), BhoomiDrishti completion (`artifacts/bhoomi-ai/src/App.tsx`).

Stage 6 — Check whether reported uncertainty matches observed errors

A confidence number is useful only if it has a defined meaning. Your calibration module asks whether

reported positional uncertainty is consistent with errors measured against reference points.

It takes examples with true positions, predicted positions and estimated uncertainties. It divides positional error by estimated uncertainty, then uses a conservative quantile of those scores to scale future uncertainty bands. It calculates separate results for formal, informal and institutional examples so good results in one group do not conceal poor results in another.

The small-sample rule can return an unbounded band when there is too little information for a finite

one. It also reports observed coverage on separate test examples. These are actual calculations.

The limits matter: this module is not connected to the ward-processing API, and its acceptance test

uses synthetic examples with older-map estimates, not a deployed trained model on actual survey

imagery. The implemented point-level calibration is not automatically a simultaneous guarantee for

an entire parcel boundary. The API explicitly returns `boundary_certification: "not_calibrated"`.

BhoomiDrishti's confidence and risk scores have no equivalent calibration path in the supplied code.

They are fixed, randomly generated or calculated from preset formulas. A displayed 98.4% does not

establish that 98.4% of boundaries lie within any stated positional tolerance.

Sources: your calibration (`geocadastra/core/conformal.py`), calibration test
(`geocadastra/tests/test_stage6_acceptance.py`), constraints endpoint (`geocadastra/api/main.py`),
BhoomiDrishti metrics (`artifacts/bhoomi-ai/src/App.tsx`).

Stage 7 — Choose which parcels a surveyor should visit first

Your prioritisation module uses shared boundaries to estimate which visit may resolve the most remaining uncertainty. If a surveyor checks a boundary shared by A and B, the result may help both

parcels. The ordering also considers where parcels are located and the time spent travelling.

The code derives adjacency from the graph, assigns a parcel the worst uncertainty among its boundary

edges, scores candidates using their own and neighbouring uncertainty, groups nearby candidates, and

orders visits. A simulation charges for travel between visited stops and for each inspection, updates

edge uncertainty, and checks whether neighbours now meet the tolerance without another visit.

The output is a curve of the fraction considered verified against simulated survey hours. The

acceptance test compares the proposed order with random ordering and an order that starts with the

least certain parcels.

This is implemented experimental planning logic, not a deployed field dispatch system. The acceptance

test synthesises edge uncertainties and assumes a visit resolves the visited parcel's boundary edges.

Its travel model is straight-line distance and a configured speed, not a road-network route or a measurement of actual crew productivity. The API does not yet expose the full planning workflow.

BhoomiDrishti has a field page and a topology queue with priority labels. The topology risks come

from templates and formulas, and entries are sorted by those values. It does not calculate the effect

of resolving a shared boundary on neighbouring parcels or simulate competing visit orders with travel

costs.

The field page's verification action patches a parcel status. It does not submit a measured coordinate.

Its offline queue count is a fixed display, and no persistent offline measurement queue or

synchronisation implementation was found. The current page also requests `status=pending`, while the

sample server parcels use `verified`, `review` and `flagged`, so that sample data supplies no pending records to the queue.

Sources: your planning (geocadastra/core/priority.py), planning test
(geocadastra/tests/test_stage7_acceptance.py), BhoomiDrishti field page
(artifacts/bhoomi-ai/src/App.tsx), sample API records (artifacts/api-server/src/routes/bhoomi.ts).

Stage 8 — Connect processing to stored jobs and API requests

Your API accepts a synthetic seed and ward dimensions, creates a ward, and stores its vector records.

It can fit older-map alignment, dispatch or resume block processing, return job status, retrieve parcels in a bounding box, list conflicts, resolve parcel associations, apply edits and survey measurements, report constraints and metrics, and return vector map tiles.

The unit of processing is one block. A transaction-scoped lock serialises competing work for that block. A successful transaction publishes its changes and status together; an exception rolls back the

attempted work and records failure. An already completed block can be skipped when a ward is resumed.

Individual unsafe fusion proposals can be refused without discarding the entire safe block attempt.

The connected path today is:

Seed and dimensions

- > generated ward and stored vector records
- > regenerated simulated boundary evidence for a block
- > raster parcel allocation
- > shared-edge graph and parcel associations
- > recorded-area refinement for a newly seeded graph
- > combination with older-map and selected survey evidence
- > validated database changes
- > final area/coverage reports and conflicts
- > status, parcel queries and map tiles

On a reprocessing path where a graph already exists, the worker retains its identity/history and reuses it rather than always seeding and refining a new graph. The training/inference, calibration and

survey-order modules sit alongside this path and still need integration.

Your application has a real Celery task and asynchronous broker configuration. However, the acceptance

tests mainly exercise eager, in-process task execution and simulated partial completion. That

establishes useful task and database behaviour, not every failure mode of a deployed broker and worker

fleet. A **done** block may also still have reported area or coverage conflicts; completion means processing ended, not that the parcel boundaries were certified.

BhoomiDrishti's API exposes dashboard, region, parcel, processing-run, topology, change and export

routes. These are callable web routes, but most return or modify sample state. Creating a processing

run inserts an object with progress 14 and has no worker that advances it. The current inference-studio

start action sets browser state to show an animation; it does not invoke a model-processing service.

The separate UI paths are also important. The state dashboard reads its GeoJSON properties; the parcel

explorer generates its own rectangles; the topology page relocates another dataset; and the field page

reads the six server parcels. These screens do not yet operate on one durable set of parcel records

produced by an inference run.

Sources: your worker (`geocadastra/jobs/orchestrator.py`), API (`geocadastra/api/main.py`), API test (`geocadastra/tests/test_stage8_acceptance.py`), BhoomiDrishti routes (`artifacts/api-server/src/routes/bhoomi.ts`), browser workflows (`artifacts/bhoomi-ai/src/App.tsx`).

What BhoomiDrishti adds beyond your present interface

Its frontend is a substantial part of the supplied project. It has navigation, region selection, map zooming and popups, an image browser, map/table parcel views, owner/identifier search, status displays,

model-comparison layouts, and review/detail screens. These make it much easier to demonstrate the

intended user experience than a backend API alone.

Its parcel explorer implements actual CSV and GeoJSON downloads using browser-created files. Those

exports contain the generated explorer parcels. This should be distinguished from its separate export

center: that screen asks the server for a ready job and a filename, but the server does not create the

advertised GeoPackage, Shapefile or signed archive.

The included image/mask collection could also be useful for future experiments, subject to checking what

its labels mean and whether geographic metadata and permitted usage are available. It cannot be dropped

into your current training pipeline unchanged: your training data expects height information and parcel-boundary targets, while the inspected assets are images and categorical masks.

Its change-detection page displays predefined examples, rather than comparing two dated image/map

inputs. Its billing page shows fixed provider, GPU, usage and cost text, rather than querying deployed

resources. Your project does not implement a full dated-image change-detection service or cloud-billing

dashboard either.

An exact comparison of four user actions

User action	Your current implementation	BhoomiDrishti's current implementation
"Process this area."	API creates a generated ward; actual geometry calculations run per block and results persist.	User selects a bundled patch and region; a timer displays stages, then a count and marker are generated from a name/random coordinates.
"Move this shared corner."	API updates a graph node through a validated changeset, checks neighbours/areas/coverage/concurrency, and records evidence.	Parcel PATCH can replace one polygon's coordinate array in server memory; topology auto-fix performs a different local union-and-hide operation.
"Verify this parcel in the field."	Request supplies ward, block, parcel, node and measured x/y; measurement and permitted movement are saved together.	Action changes status to verified ; no measured point is part of that action.
"Show confidence and export."	Separate positional calibration exists, but ward API reports not calibrated; spatial queries and vector tiles use saved geometry.	Screens display generated scores; explorer can download generated data, while export-center package generation is only a filename response.

What remains before your project can support the full workflow being shown

1. Fix the area-tolerance optimisation defect and retain a regression that demonstrates a feasible, unequal-total allocation.
2. Connect a versioned trained model to the worker so real predictions replace simulated evidence. Record which weights and inputs produced each result.
3. Add real raster/vector ingestion, including coordinate systems, image-to-map transforms, height data where required, and persistent raster storage. A PNG filename and a randomly placed marker do not provide that information.
4. Validate model accuracy on appropriately separated real examples, especially where roofs, walls and parcel boundaries differ. More compute makes larger experiments possible; it does not establish their outcome.
5. Connect calibration to the actual predictions, preserve separate measurement roles, and invalidate or recompute affected results after edits. Define precisely what is guaranteed at a point, edge or whole-parcel level.
6. Expose the survey-planning output and build a real measurement-capture workflow, including durable offline storage if it is required.
7. Build an interface around the saved parcel IDs, geometries, conflicts and job states. BhoomiDrishti provides examples of useful views, but its sample-data generators and simulated metrics should not become your source of truth.
8. Implement authentication and permissions, real worker/broker failure tests, operational monitoring, scale measurements and any required GIS export formats.

For presenting the current work, a precise description of GeoCadastra is: **"We have built and tested the parcel-geometry and record-processing backend on synthetic wards. It shares boundaries between parcels, uses recorded areas during reconstruction, validates edits, and preserves history. Training, calibration and survey-order code exist, but real-data validation and full integration remain."** A precise description of the supplied BhoomiDrishti folder is: **"It provides a broad interactive interface and bundled map/image assets. Several displayed processing, accuracy and repair results are simulated; the underlying parcel-processing and storage service remains largely to be implemented."**

Part III — Verification log

Each row records a claim from Part II and the independent check performed on 14 September 2026.

"Executed" means code was run; "counted" means files or records were enumerated; "read" means the

cited source line was located and inspected.

Claim checked	Method	Result
capacity.py rejects a feasible 33/33/34 allocation	Executed solver from 20/20/60	True. converged: False, recorded_areas_not_met_after_snapping, max_error_m2: 2.0

Claim checked	Method	Result
Same function accepts the feasible arrangement	Executed solver from 33/33/34	True. <code>converged: True, already_within_tolerance</code>
Model metrics derived from a name hash	Executed <code>computeModelMetrics</code> verbatim	True. <code>file_that_does_not_exist</code> returns IoU 0.883, 197 ms
690 image / 690 mask / 690 binary PNGs	Counted directory entries	True. 690 in each of the three folders
<code>parcels_21.geojson</code> holds 300 polygons	Parsed and counted features	True. 300 features
Administrative boundary file holds 37 features	Parsed and counted features	True. 37 features
Composite boundary file holds one feature	Parsed and counted features	True. 1 feature
Explorer requests 300 generated parcels	Read call site	True. <code>generateMicroParcels(bounds, code, 300)</code>
Field page queries a status the API never returns	Read both sides	True. Page requests <code>status: 'pending'</code> ; API parcels are verified/review/flagged only
Auto-fix unions two polygons and clears every error	Read implementation	True. Unions features 0 and 1, hides one, sets <code>fixed: errors.length</code> , then <code>setErrors([])</code>
Auto-fix is timer-driven	Read implementation	True. <code>setTimeout(..., 1200)</code>
Overlap entry can be created with an empty intersection	Read implementation	True. Entry pushed regardless; falls back to <code>poly1</code> centre/bbox
Seven model checkpoints listed, none loaded	Read <code>UNET_MODELS</code> and usage	True. 7 entries; no weight loading in that path
Model outputs differentiated by CSS filters	Read <code>modelImageFilter</code>	True. Seeded brightness/contrast/saturate on the same mask
Binary category PNG presented as a DTM layer	Read UI labels	True. "DTM Binary — Bare-Earth vs Structure Separation"
Confidence and IoU are hardcoded on completion	Read completion handler	True. <code>confidence: 98.4, iou: 0.887</code> literals
Parcel count derived from patch-name characters	Read derivation	True. <code>patchId.split('').reduce(... charCodeAt...) % 400 + 900</code>
Completion marker randomly placed in a state bbox	Read derivation	True. <code>minLat + Math.random() * (maxLat - minLat)</code>
Export center returns a filename, not a package	Read route	True. Returns id/format/status/fileName only

Claim checked	Method	Result
Ed25519 label with no signing implementation	Searched application path	True. Label present; no signing code found
Database schema exports no application tables	Read schema file	True. Template comments and <code>export {}</code>
No application test suite	Searched for test files and scripts	True. No test file, runner or script found
GeoCadastra uses a SHA-256 provenance chain	Read provenance module	True. <code>hashlib.sha256(...).hexdigest()</code>
"SDF" target is a nonnegative distance	Read dataset target construction	True. <code>distance_transform_edt(~boundary_mask) * gsd</code>
API reports <code>boundary_certification: "not_calibrated"</code>	Read API and its test	True. Present in both
Focused suite returns 60 passed, 1 deselected	Executed the cited command	True. 60 passed, 1 deselected in 8.09 s
Fast suite passes	Executed full fast suite earlier this session	True. 394 passed
Saved full-suite log reports 427 passed, 1 xfail	Read stored log	True. Matches

The focused test command, reproduced verbatim:

```
OMP_NUM_THREADS=2 MKL_NUM_THREADS=2 .venv/bin/python -m pytest \
  geocadastra/tests/test_conformal.py \
  geocadastra/tests/test_priority.py \
  geocadastra/tests/test_subdivision_precision.py \
  geocadastra/tests/test_capacity_refinement.py \
  -k 'not worker' -q --tb=short
-> 60 passed, 1 deselected, 11 warnings in 8.09s
```

The capacity-solver reproduction, abbreviated:

```
block = 10m x 10m (100 m2), records {1: 34.0, 2: 34.0, 3: 34.0}, tolerance +/- 1.0 each
start 20/20/60 -> converged: False
                  reason: recorded_areas_not_met_after_snapping
                  max_error_m2: 2.0
                  optimizer_message: "Optimization terminated successfully"
start 33/33/34 -> converged: True
                  reason: already_within_tolerance
```

The fabricated-metric reproduction, abbreviated:

```
patch_1067          -> iou 0.903  dice 0.955  loss 0.0521  inferenceMs 167
file_that_does_not_exist-> iou 0.883  dice 0.975  loss 0.0620  inferenceMs 197
zzzz-not-a-patch    -> iou 0.897  dice 0.960  loss 0.0632  inferenceMs 109
```

Limits of this verification

The other project's dependencies were not installed, so its browser and server stack were not

launched; its behaviour was traced from source and by executing its pure functions in isolation.

The full GeoCadastra slow suite was not re-run for this document; the figure quoted is from the run completed earlier in this session and from the repository's stored log. Claims about external data provenance, legal ownership, and real-world survey accuracy were outside what either repository can

establish on its own.